running

From Discrete Logarithms to Curve25519 and X25519

aadarwal-training-data

Abstract

This note builds the shortest rigorous path from modular arithmetic to X25519. It defines the discrete logarithm problem in an arbitrary cyclic group, instantiates it as the elliptic-curve discrete logarithm problem, derives the Diffie–Hellman equality, and then studies Curve25519's field, subgroup, cofactor, quadratic twist, x-only Montgomery ladder, and byte encoding. The last section states the boundary between a scalar-multiplication primitive and a secure authenticated protocol. Companion marimo notebooks make every small example executable and visual.

Contents

1 Finite arithmetic and cyclic groups

1.1 Congruence and finite fields

For an integer $n > 1$, write $a \equiv b \pmod{n}$ when n divides $a - b$. The residue classes form the ring $\mathbb{Z}/n\mathbb{Z}$. Addition and multiplication are performed normally and then reduced to a representative modulo n.

Definition 1.1 (Finite prime field)**.** *For a prime p, the field*

$$\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$$

has p elements. Every nonzero $a \in \mathbb{F}_p$ has a unique inverse a^{-1} satisfying $aa^{-1} = 1$.

Primality matters. If $n = rs$ is composite, then the nonzero classes of r and s multiply to zero, so neither can have an inverse. Conversely, Bezout's identity gives an inverse for every nonzero class modulo a prime. Thus $\mathbb{Z}/n\mathbb{Z}$ is a field exactly when n is prime.

Division in $\mathbb{F}_p$ means multiplication by an inverse. The extended Euclidean algorithm computes that inverse, while Fermat's little theorem provides the identity

$$a^{-1} = a^{p-2} \pmod{p} \qquad (a \neq 0).$$

The latter is particularly useful when an implementation already has an efficient fixed-pattern exponentiation routine.

The nonzero elements form the multiplicative group

$$\mathbb{F}_p^{\times} = \mathbb{F}_p \setminus \{0\}, \qquad |\mathbb{F}_p^{\times}| = p - 1.$$

Do not confuse this group with the field's additive group, whose identity is 0 rather than 1.

1.2 Groups, generators, and order

Definition 1.2 (Group)**.** *A group $(\mathcal{G}, \circ)$ has an associative binary operation, an identity e, and an inverse for every element. It is abelian when $x \circ y = y \circ x$.*

For $g \in \mathcal{G}$, the cyclic subgroup generated by g is

$$\langle g \rangle = \{g^k : k \in \mathbb{Z}\}.$$

Its size is the order $\mathrm{ord}(g)$, equivalently the smallest positive r for which $g^r = e$. In additive notation the same statements become

$$\langle G \rangle = \{[k]G : k \in \mathbb{Z}\}, \qquad \mathrm{ord}(G) = \min\{r > 0 : [r]G = \mathcal{O}\}.$$

Square brackets show that scalar multiplication is repeated group addition; it is not multiplication of the coordinates.

Lagrange's theorem says the order of a subgroup divides the order of the finite ambient group. Consequently, cryptographic parameter design is driven by the factorization of the group order, not merely by the number of bits in the field modulus.

A running multiplicative example. In $\mathbb{F}_{29}^{\times}$, the element 2 has order 28 and generates the whole group. Forward evaluation quickly gives

$$2^{19} \equiv 26 \pmod{29}.$$

The inverse question "which exponent produced 26?" is a tiny instance of the discrete logarithm problem. The first marimo notebook makes the orbit and all inverses visible.

1.3 Quadratic residues

A nonzero $r \in \mathbb{F}_p$ is a quadratic residue when $r = y^2$ for some y. Euler's criterion packages this test as the Legendre symbol

$$\left(\frac{r}{p}\right) = r^{(p-1)/2} \pmod{p} \in \{1, -1\}.$$

Exactly half of the nonzero field elements are squares. Later, this one-bit classification tells us whether a Montgomery u-coordinate lifts to a point on a curve or on its quadratic twist.

> **Key point. Keep three integers separate: the field prime p, the order ℓ of the chosen prime-order subgroup, and the cofactor h satisfying $\#\mathcal{G} = h\ell$.**

2 The discrete logarithm problem

2.1 Formal definition

Definition 2.1 (Discrete logarithm problem)**.** *Let $\mathcal{G} = \langle g \rangle$ be a cyclic group of order r. Given g and $h \in \mathcal{G}$, the discrete logarithm problem is to find*

$$x \in \mathbb{Z}/r\mathbb{Z} \quad \textit{such that} \quad h = g^x.$$

In additive notation, given G and $Q \in \langle G \rangle$, find x such that

$$Q = [x]G.$$

The group, generator, and generator order are part of the problem statement. The answer is unique only modulo $\mathrm{ord}(g)$. If the target is outside the generated subgroup, no answer exists.

2.2 What "efficient" and "hard" mean

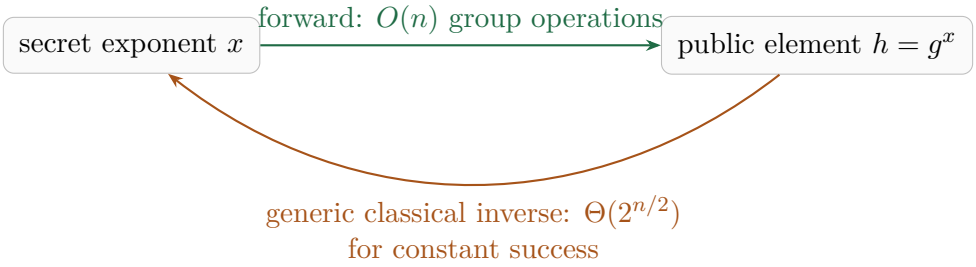
Write

$$n = \lceil \log_2 r \rceil,$$

the bit length of the exponent space. Complexity must be measured in n, not in the numerical value r. After reducing x modulo r, binary square-and-multiply uses $n-1$ squarings and at most $n-1$ additional multiplications. Thus

$$x \longmapsto g^x \qquad \text{costs fewer than } 2n \text{ group operations.}$$

Double-and-add gives the same $O(n)$ count for $x \mapsto [x]G$. A group operation is not a single hardware instruction: modular multiplication and reduction have their own bit cost. If field elements have k bits and $M(k)$ is the bit cost of a k-bit multiplication, finite-field square-and-multiply costs roughly $O(nM(k))$ up to reduction factors. This is still polynomial in the encoded input length.



The reverse arrow combines an upper bound with a restricted lower bound. Baby-step–giant-step and Pollard rho use $O(\sqrt{r}) = O(2^{n/2})$ group operations. More precisely, Shoup bounded the success probability of a classical generic attack using T oracle operations by $O(T^2/r)$ when r is prime [12]. Constant success therefore requires $\Omega(\sqrt{r})$ operations, and the generic classical complexity is $\Theta(2^{n/2})$.

Problem	Known work	Class	Status
$x \mapsto g^x$ or $[x]G$	$O(n)$ group operations	polynomial	proven algorithm
generic DLP/ECDLP	$\Theta(2^{n/2})$	exponential	generic-model theorem
chosen concrete curve	best known: generic	exponential	assumption; no unconditional bound
quantum DLP/ECDLP	poly(n)	polynomial	Shor's algorithm

The word *generic* is essential. The lower bound deliberately ignores the internal representation of a group element, so it cannot rule out a shortcut that exploits a particular encoding. No unconditional

theorem says that Curve25519 ECDLP requires exponential classical time. Its hardness is an assumption supported by the generic lower bound, decades of cryptanalysis, and the absence of a curve-specific shortcut.

Complexity-theoretic cryptography states that assumption over a family of groups, and for random instances rather than one hand-picked puzzle:

Definition 2.2 (Classical DLP hardness assumption)**.** *Let $(\mathcal{G}_\lambda = \langle g_\lambda \rangle)$ be an efficiently represented family of prime-order groups, with orders r_λ. DLP is hard for the family if every classical probabilistic polynomial-time algorithm $\mathcal{A}$ satisfies*

$$\Pr_{x \leftarrow \mathbb{Z}/r_\lambda\mathbb{Z}} [\mathcal{A}(\mathsf{par}_\lambda, g_\lambda, g_\lambda^x) \equiv x \pmod{r_\lambda}] \leq \mathrm{negl}(\lambda).$$

The probability includes the randomness of x and of $\mathcal{A}$; negl *decreases faster than every inverse polynomial.*

Within one fixed cyclic group, DLP is random self-reducible. Given an arbitrary target $h = g^x$, choose random t and send $hg^t = g^{x+t}$ to a solver for random targets; subtracting t from its answer recovers x. This connects random-target and worst-target instances inside that group, but it does not prove that the selected group family is hard.

This is an average-case one-wayness assumption, not a proof that DLP is NP-hard. Such NP-hardness is not known, and worst-case NP-hardness alone would not provide the random-instance hardness needed in cryptography. Curve25519 is one fixed parameter set, so its "security bits" are a concrete estimate of the best known attack cost rather than an asymptotic theorem.

2.3 Generic attacks

Method	Time	Storage
exhaustive search	$O(r) = O(2^n)$	$O(1)$
baby-step–giant-step	$O(\sqrt{r}) = O(2^{n/2})$	$O(\sqrt{r})$
Pollard rho	expected $O(\sqrt{r}) = O(2^{n/2})$	$O(1)$
Pohlig–Hellman	controlled by the largest prime factor of r	varies

Baby-step–giant-step writes $x = im + j$ with $m = \lceil \sqrt{r} \rceil$ and looks for a collision

$$h(g^{-m})^i = g^j.$$

Pollard rho replaces the stored table by a pseudorandom walk and a cycle-finding method [10]. The classical generic-group lower bound explains why the square root repeatedly appears [12].

Pohlig–Hellman reduces a DLP of order $r = \prod_i q_i^{e_i}$ to DLPs in the prime-power factors [9]. A secure group therefore needs a large prime factor; a large ambient field by itself is not enough.

Finite-field multiplicative groups also admit index-calculus-style algorithms that exploit their representation and beat the generic square-root bound. For prime fields, the number field sieve has heuristic subexponential running time of the form [4]

$$L_p[1/3, c] = \exp\Big((c + o(1))(\log p)^{1/3}(\log\log p)^{2/3}\Big).$$

This is subexponential in p but still superpolynomial in the encoded input length $\log p$. No analogous general subexponential method is known for a properly chosen elliptic-curve group. This is why a 255-bit elliptic-curve field can target a security level that would require a much larger finite-field Diffie–Hellman modulus.

All of these classical comparisons have a quantum boundary. Shor's algorithm solves DLP, including ECDLP, in time polynomial in the encoded input size on a sufficiently capable fault-tolerant quantum computer [11]. The hardness assumption is therefore explicitly classical, not post-quantum.

2.4 DLP, CDH, and DDH are different problems

Let g generate a group of prime order r and choose unknown $a, b \in \mathbb{Z}/r\mathbb{Z}$.

Definition 2.3 (Computational Diffie–Hellman)**.** *The* CDH *problem maps*

$$(g, g^a, g^b) \longmapsto g^{ab}.$$

In additive notation it maps $(G, [a]G, [b]G)$ *to* $[ab]G$.

Definition 2.4 (Decisional Diffie–Hellman)**.** *The* DDH *problem asks whether a supplied* T *equals* g^{ab}*, given* (g, g^a, g^b, T).

A DLP solver gives a CDH solver: recover a, then compute $(g^b)^a$. A CDH solver gives a DDH solver: compute g^{ab} and compare it with T. Thus there are solver implications

$$\mathsf{DLP} \Longrightarrow \mathsf{CDH} \Longrightarrow \mathsf{DDH},$$

so the implications between hardness assumptions run in the reverse direction:

$$\mathsf{DDH}\text{ hard} \Longrightarrow \mathsf{CDH}\text{ hard} \Longrightarrow \mathsf{DLP}\text{ hard}.$$

These problems are not known to be equivalent in arbitrary groups. In some pairing-friendly groups, for example, DDH is easy while CDH may remain hard. Saying "DLP is hard" is therefore not itself a proof of every Diffie–Hellman assumption.

2.5 From one-wayness to cryptography

For a prime-order group with a full point encoding, $x \mapsto g^x$ is a bijection and a candidate *one-way permutation*: everyone can evaluate it, while inversion on a random output is assumed hard. This easy-forward, hard-backward pattern is one of cryptography's recurring building blocks. Hash functions seek preimage resistance, and trapdoor constructions add secret information that enables inversion.

Diffie–Hellman uses the asymmetry differently: neither Alice nor Bob inverts the public map. Each applies a scalar they already know to the other party's public element. A passive outsider must compute the shared element without knowing either scalar, which is the CDH problem. Thus one-wayness is necessary intuition but not a complete security argument: key agreement needs a CDH-style assumption and a full protocol still needs key derivation, authentication, and message protection.

> **Key point. Curve25519 does not introduce a new kind of logarithm. It supplies a carefully chosen cyclic subgroup in which the ordinary DLP is written additively and is called ECDLP.**

3 Elliptic-curve groups and ECDH

3.1 From a cubic equation to a group

For an odd prime p, a short Weierstrass curve has the form

$$E: \quad y^2 = x^3 + ax + b, \qquad 4a^3 + 27b^2 \not\equiv 0 \pmod{p}.$$

The nonzero discriminant excludes cusps and self-intersections. The solutions in $\mathbb{F}_p^2$, together with a formal point at infinity $\mathcal{O}$, form a finite abelian group. The familiar chord-and-tangent picture over the reals is useful intuition; over $\mathbb{F}_p$ the curve is a discrete point cloud and the algebraic formulas are what survive.

Curve25519 uses Montgomery form [8]:

$$E_{A,B}: \quad Bv^2 = u^3 + Au^2 + u, \qquad B(A^2 - 4) \neq 0.$$

For $B = 1$ and distinct noninverse points $P = (u_1, v_1)$ and $Q = (u_2, v_2)$, put

$$\lambda = (v_2 - v_1)(u_2 - u_1)^{-1}.$$

Then $P + Q = (u_3, v_3)$ where

$$u_3 = \lambda^2 - A - u_1 - u_2, \qquad v_3 = \lambda(u_1 - u_3) - v_1.$$

For doubling, replace the slope by

$$\lambda = (3u_1^2 + 2Au_1 + 1)(2v_1)^{-1}.$$

All operations occur in $\mathbb{F}_p$. The exceptional cases complete the group law: $P + \mathcal{O} = P$, and $P + (-P) = \mathcal{O}$ with $-(u, v) = (u, -v)$.

3.2 Scalar multiplication and ECDLP

Double-and-add evaluates $[k]P$ from the binary expansion of k using $O(\log k)$ additions. The corresponding elliptic-curve discrete logarithm problem is

$$(E, G, Q = [a]G) \longmapsto a \pmod{\operatorname{ord}(G)}.$$

It is exactly the DLP from the preceding section, instantiated in an elliptic curve subgroup.

Hasse's theorem bounds the ambient group order:

$$\left|\#E(\mathbb{F}_p) - (p + 1)\right| \leq 2\sqrt{p}.$$

Cryptographic parameters select a point G of large prime order ℓ and write

$$\#E(\mathbb{F}_p) = h\ell,$$

where h is the cofactor. Multiplying an arbitrary point by h moves it into a subgroup whose order divides ℓ, but protocol-specific validation rules are still required.

3.3 An exact toy Montgomery curve

The notebooks use

$$E/\mathbb{F}_{127}: \quad v^2 = u^3 + 5u^2 + u.$$

It has $136 = 8 \cdot 17$ points. The point

$$G = (18, 55)$$

has order 17, so its subgroup is small enough to enumerate yet has the same "cofactor times prime" shape as Curve25519. For example,

$$[5]G = (122, 73), \qquad [9]G = (124, 74), \qquad [11]G = (70, 64).$$

This curve is pedagogical and cryptographically worthless.

3.4 Elliptic-curve Diffie–Hellman

Alice chooses a and publishes $A = [a]G$. Bob chooses b and publishes $B = [b]G$. Associativity of scalar multiplication gives

$$[a]B = [a]([b]G) = [ab]G = [b]([a]G) = [b]A.$$

Alice and Bob never run the hard inverse. They compute *sideways*: each combines a scalar they know with the public point they received.

$$\begin{aligned} \text{Alice: } (a, B) &\longmapsto [a]B = [ab]G, \\ \text{Bob: } (b, A) &\longmapsto [b]A = [ab]G, \\ \text{Eve: } (G, A, B) &\not\longmapsto [ab]G \qquad \text{(the CDH challenge).} \end{aligned}$$

The secret scalar is not a trapdoor that makes inversion easy; it enables a different efficient group action that both honest parties can make agree.

For the toy defaults $a = 5$ and $b = 9$, both sides reach

$$[45]G = [11]G = (70, 64),$$

because scalars are interpreted modulo 17.

The original Diffie–Hellman insight is that public values can be combined with a private scalar to reach the same hidden group element [3]. A passive observer receives G, A, B and faces CDH; recovering either scalar via ECDLP is one sufficient way to solve it.

4 Curve25519 and its discrete logarithm

Bernstein designed Curve25519 for fast Diffie–Hellman and straightforward constant-time implementations [2]. RFC 7748 gives the modern interoperable specification [7].

4.1 Parameters

Field prime	$p = 2^{255} - 19$
Curve	$v^2 = u^3 + 486662u^2 + u$ over $\mathbb{F}_p$
Prime subgroup order	$\ell = 2^{252} + 27742317777372353535851937790883648493$
Cofactor	$h = 8$, so $\#E(\mathbb{F}_p) = 8\ell$
Base point	Montgomery coordinate $u(G) = 9$, with $\mathrm{ord}(G) = \ell$

The name "25519" comes from $2^{255} - 19$, not from a catalog number or a claim of 255-bit security. The pseudo-Mersenne shape gives

$$2^{255} \equiv 19 \pmod{p},$$

so high limbs can be folded back into the field efficiently. The public coefficient 486662 is not a key: it is part of a parameter choice that gives useful group and twist orders together with efficient Montgomery formulas.

Since $\ell \approx 2^{252}$, a generic Pollard-rho attack costs on the order of

$$\sqrt{\ell} \approx 2^{126}$$

group operations. The curve is therefore conventionally placed near the 128-bit classical security level. Field size and security level are different quantities.

The forward and inverse costs are now concrete. X25519 evaluates a public key with a fixed 255-round Montgomery ladder, each round using a constant number of field operations. For an honest prime-subgroup public key, the best known general classical inversion has the scale

$$\begin{aligned} a &\longmapsto u([a]G) && \text{255 ladder rounds,} \\ u([a]G) &\longmapsto \{a, -a\} \pmod{\ell} && \text{about } 2^{126} \text{ generic curve operations.} \end{aligned}$$

Ladder rounds and Pollard-rho steps are not identical units of wall-clock work. The point of the comparison is their scaling: fixed polynomial-sized forward work versus exponential generic inversion in the subgroup bit length. The x-only sign ambiguity changes what counts as an inverse, but not the asymptotic attack cost. The displayed 2^{126} is an order-of-magnitude, not an exact wall-clock guarantee: the x-only quotient, clamped-scalar interval, parallelism, and constant factors move concrete estimates within roughly the 2^{125}–2^{126} range [2].

4.2 What changes when only the u-coordinate is exposed?

On a Montgomery curve,

$$-(u, v) = (u, -v).$$

Thus $u(P) = u(-P)$: retaining only u identifies a point with its inverse. For the odd prime-order subgroup $\langle G \rangle$, this is the only ambiguity.

Proposition 4.1. *For nonidentity points in $\langle G \rangle$,*

$$u([a]G) = u([b]G) \quad \Longleftrightarrow \quad b \equiv a \text{ or } b \equiv -a \pmod{\ell}.$$

Proof. Two nonidentity curve points with the same u have v-coordinates that are negatives, so they are equal or inverses. Hence $[b]G = [a]G$ or $[b]G = -[a]G$. Since G has order ℓ, the corresponding scalar congruences follow. The reverse implication is immediate. □

Consequently, the full-point map is a candidate one-way permutation on the prime-order subgroup, while the x-only map is two-to-one away from the identity. Its discrete-log relation for an honest public key is recovery of a up to sign modulo ℓ. That is enough to break Diffie–Hellman, because

$$u([-a]B) = u(-[a]B) = u([a]B).$$

An ECDLP solution recovers a scalar residue, not necessarily the original 32-byte secret input: X25519 deliberately overwrites five input bits during scalar decoding, and scalar multiplication depends only on the residue in the subgroup.

The corresponding x-only CDH problem is

$$(u(G), u([a]G), u([b]G)) \longmapsto u([ab]G).$$

This is the formal connection between the abstract DLP and honest Curve25519 public keys. It does not establish an equivalence between ECDLP and CDH; it shows that an ECDLP solver is sufficient to solve the Curve25519 CDH instance.

4.3 The quadratic twist and cofactor

For a nonsquare $d \in \mathbb{F}_p$, a quadratic twist can be represented as

$$E^d : \quad dv^2 = u^3 + Au^2 + u.$$

The curve and twist orders satisfy

$$\#E(\mathbb{F}_p) + \#E^d(\mathbb{F}_p) = 2(p+1).$$

A received u may correspond to the main curve or to its twist. Curve25519's twist has cofactor 4 and a large prime factor, so the x-only ladder can be defined robustly on both sides. Inputs in a small-order component can still produce the all-zero output. Making the scalar divisible by 8 annihilates the curve's cofactor-8 component (and the twist's cofactor-4 component), but it does not make every peer input contributory or authenticated.

> **Key point. The hard group problem lives in the subgroup of order ℓ, not in all 2^{255} possible byte strings and not in the cofactor-8 ambient group.**

5 X25519 as an x-only byte-string function

Curve25519 is a curve. X25519 is the RFC-defined function

$$\{0,1\}^{256} \times \{0,1\}^{256} \longrightarrow \{0,1\}^{256}$$

that decodes a scalar and a Montgomery u-coordinate, evaluates x-only scalar multiplication, and returns a 32-byte u-coordinate [7].

5.1 Scalar and coordinate decoding

The scalar is a 32-byte little-endian string. Before interpreting it as an integer, X25519 applies the bit operation commonly called *clamping*:

```
k[0] &= 248; /* clear bits 0, 1, and 2 */
k[31] &= 127; /* clear bit 255 */
k[31] |= 64; /* set bit 254 */
```

The resulting integer is exactly

$$k = 2^{254} + 8m, \qquad 0 \le m \le 2^{251} - 1.$$

It is divisible by 8, its top processed bit is fixed, and it is not simply the original bytes reduced modulo ℓ. Clamping cannot create entropy.

An input u is also decoded little-endian. X25519 masks the most significant bit of the final byte and accepts noncanonical values, processing them as field elements modulo p. Generic "full point validation" is not the X25519 input rule: no v-coordinate is supplied, and the ladder intentionally handles both the curve and its twist.

5.2 The Montgomery ladder

Conceptually, after processing a scalar prefix r, the ladder maintains adjacent multiples

$$R_0 = [r]P, \qquad R_1 = [r+1]P.$$

For the next bit, the abstract transition is

bit	R_0'	R_1'
0	$[2]R_0$	$R_0 + R_1$
1	$R_0 + R_1$	$[2]R_1$.

One doubling and one differential addition occur for either bit. Constant-time conditional swaps select the registers without a secret-dependent branch.

The implementation stores projective coordinates $X : Z$ representing $u = X/Z$. This defers the costly field inversion until the end. Let x_1 be the input u and let (X_2, Z_2) and (X_3, Z_3) represent the two ladder registers. One RFC ladder step uses

$$\begin{aligned} A_0 &= X_2 + Z_2, & AA &= A_0^2, \\ B_0 &= X_2 - Z_2, & BB &= B_0^2, \\ E &= AA - BB, & & \\ C_0 &= X_3 + Z_3, & D_0 &= X_3 - Z_3, \\ DA &= D_0 A_0, & CB &= C_0 B_0, \end{aligned}$$

followed by

$$\begin{aligned} X_3' &= (DA + CB)^2, & Z_3' &= x_1 (DA - CB)^2, \\ X_2' &= AA\,BB, & Z_2' &= E(AA + a_{24} E), \end{aligned}$$

where

$$a_{24} = \frac{486662 - 2}{4} = 121665.$$

All expressions are reduced modulo p. Another common, equivalent convention uses $121666 = (A+2)/4$ together with BB in the last formula; mixing the constant from one convention with the formula from the other is a bug.

After scanning bits 254 through 0, the output is

$$u = X_2 Z_2^{p-2} \pmod{p},$$

encoded as 32 little-endian bytes. Fermat inversion also maps $Z_2 = 0$ to the all-zero output in the RFC computation.

The fixed arithmetic pattern removes a common timing leak, but regularity is not a proof against every cache, power, fault, or microarchitectural attack. The companion Python follows the formulas for inspection and test vectors; its branches and big integers are explicitly variable-time.

5.3 X25519 Diffie–Hellman

The base coordinate is encoded as the byte 9 followed by 31 zero bytes. Alice and Bob compute

$$A = \mathrm{X25519}(a, 9), \qquad B = \mathrm{X25519}(b, 9),$$
$$S_A = \mathrm{X25519}(a, B), \qquad S_B = \mathrm{X25519}(b, A).$$

For honest inputs, $S_A = S_B$ because both encode $u([ab]G)$. RFC 7748 recommends deriving application keys from the shared output and both public keys; an enclosing protocol must also specify how an all-zero output is handled.

6 From X25519 to a secure protocol

X25519 is neither encryption nor authentication. It computes raw shared key material. An active attacker can substitute public keys and complete two independent exchanges:

$S_{AM} \neq S_{MB}$ in general

Secrecy against a passive observer is not proof of the peer's identity. Authentication must come from a certificate and signature, a pre-shared key, an authenticated static key, or another mechanism defined by the protocol.

6.1 Key derivation and context binding

A raw Diffie–Hellman coordinate is not an application key. A construction such as HKDF first extracts a pseudorandom key and then expands labeled keys for specific purposes [6]:

$$\text{raw DH} \longrightarrow \text{all-zero handling} \longrightarrow \text{HKDF}(S, A, B, \text{roles}, \text{transcript}) \longrightarrow \begin{cases} K_{A\to B}, \\ K_{B\to A}. \end{cases}$$

Length prefixes or another unambiguous encoding prevent distinct transcripts from collapsing to the same KDF input. Ordered public keys and role labels stop reflection and unknown-key-share confusions. Separate directional keys simplify nonce and replay reasoning.

RFC 7748 permits a generic implementation to check for all-zero output, while newer profiles can strengthen that rule. The X25519-based HPKE suite requires the recipient to abort on an all-zero DH result and uses labeled HKDF inputs [1]. A high-level library should surface an unacceptable-peer-key error rather than silently continuing.

After key derivation, an AEAD supplies confidentiality and ciphertext integrity. Its nonce requirements remain independent protocol obligations.

6.2 Forward secrecy, signatures, and quantum attacks

Fresh ephemeral X25519 keys can provide forward secrecy when the handshake is authenticated and the ephemeral secrets are erased. Curve25519 itself does not guarantee freshness or erasure.

Ed25519 is a signature scheme over a related Edwards representation [5]. It is not X25519. Its public-key encoding, private-key derivation, and security purpose differ. Conversion routines exist in some libraries, but distinct purpose-specific key pairs are the safer default.

Finally, Curve25519 is not post-quantum. A sufficiently capable quantum computer running Shor's algorithm would solve ECDLP [11]. Hybrid deployments combine classical authentication and X25519 with a standardized post-quantum mechanism according to a reviewed protocol; ad hoc concatenation is not a protocol.

6.3 Composition checklist

For production work:

1. use a maintained X25519 implementation rather than handwritten field arithmetic;
2. follow the enclosing protocol's all-zero rule;
3. bind ordered public keys, roles, suite identifiers, and the transcript into a KDF;
4. authenticate the peer;
5. derive separate keys by purpose and direction;
6. use an AEAD and obey its nonce limits;
7. prefer a reviewed construction such as TLS, Noise, or HPKE.

> **Key point. The chain is modular: efficient scalar multiplication supplies the public operation; a CDH-style assumption says an outsider cannot combine two public points into their shared point; solving ECDLP is one sufficient**

break. X25519 packages this group action, a KDF turns its output into keys, authentication binds those keys to identities, and an AEAD protects messages.

7 Exercises and compact answers

1. In $\mathbb{F}_{29}$, verify that $7^{-1} = 25$ and compute 2^{19}.
2. State the DLP with all required parameters. Why is the answer defined modulo $\mathrm{ord}(g)$ rather than modulo a field prime?
3. Let $r \approx 2^n$. Compare the group-operation cost of forward exponentiation with a generic classical DLP attack. Evaluate the exponents at $n = 252$, and identify which lower bound is model-specific.
4. Why do Alice and Bob not solve a discrete logarithm during ECDH?
5. Show algebraically that a DLP solver gives a CDH solver. Explain why this does not prove the converse.
6. On the toy curve $v^2 = u^3 + 5u^2 + u$ over $\mathbb{F}_{127}$, verify that $G = (18, 55)$ lies on the curve and that $[17]G = \mathcal{O}$.
7. With toy secrets $a = 5$ and $b = 9$, compute the two public points and the shared point.
8. Why does an x-only public key recover a prime-subgroup scalar only up to sign? Why is that ambiguity irrelevant to x-only Diffie–Hellman?
9. Distinguish the field prime p, subgroup order ℓ, and cofactor h for Curve25519. Which controls generic ECDLP cost?
10. Apply the X25519 clamping rules to 32 bytes of `ff`. Which properties of the resulting integer are forced?
11. Explain why clearing the low three scalar bits does not remove the need to handle all-zero output.
12. Draw a man-in-the-middle exchange and list the inputs that should be bound into the KDF.

Answers

1. $7 \cdot 25 = 175 \equiv 1 \pmod{29}$ and $2^{19} \equiv 26 \pmod{29}$.
2. Specify a cyclic group $\mathcal{G} = \langle g \rangle$, its operation, generator g, order $r = \mathrm{ord}(g)$, and target $h \in \langle g \rangle$; solve $g^x = h$. Exponents represent the same action exactly modulo r.
3. Forward evaluation is $O(n)$ group operations; generic classical inversion is $\Theta(2^{n/2})$. At $n = 252$ the scales are hundreds of forward operations versus about 2^{126} generic attack operations. The matching $\Omega(2^{n/2})$ lower bound holds in the classical generic group model, not unconditionally for every curve encoding.
4. Alice computes $[a]B$ using her known a, while Bob computes $[b]A$ using his known b. Both are forward scalar multiplications and equal $[ab]G$; an outsider without either scalar faces CDH.
5. Recover a from (g, g^a) and compute $(g^b)^a = g^{ab}$. No known generic reduction recovers a discrete logarithm from an arbitrary CDH solver.
6. Substitution gives $55^2 \equiv 18^3 + 5 \cdot 18^2 + 18 \pmod{127}$. Double-and-add produces the identity at scalar 17; the notebook shows the entire cycle.
7. $[5]G = (122, 73)$, $[9]G = (124, 74)$, and both sides compute $[45]G = [11]G = (70, 64)$.
8. Points P and $-P$ share u. Thus the scalar is a or $-a$ modulo ℓ. Multiplying by either produces shared points that are negatives, which again share the same output u.

9. $p = 2^{255} - 19$ defines field arithmetic, $\ell \approx 2^{252}$ is the prime subgroup order, and $h = 8$ satisfies $\#E = 8\ell$. Generic ECDLP cost is governed by $\sqrt{\ell}$.
10. The first byte becomes `f8`; the final byte becomes `7f`. The integer is divisible by 8, bit 254 is set, and bit 255 is clear.
11. A malicious small-order input is annihilated by the cofactor-divisible scalar and can therefore produce zero. Clamping controls the scalar; it does not authenticate or make the peer contributory.
12. Mallory substitutes one public key toward Alice and another toward Bob, obtaining two secrets. A reviewed KDF input binds the raw DH result, ordered public keys, roles, suite or protocol label, and transcript.

References

[1] Richard Barnes, Karthikeyan Bhargavan, Benjamin Lipp, and Christopher A. Wood. Hybrid public key encryption. Technical Report RFC 9180, Internet Research Task Force, 2022.

[2] Daniel J. Bernstein. Curve25519: New diffie–hellman speed records. In *Public Key Cryptography – PKC 2006*, pages 207–228, 2006.

[3] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.

[4] Daniel M. Gordon. Discrete logarithms in GF(p) using the number field sieve. *SIAM Journal on Discrete Mathematics*, 6(1):124–138, 1993.

[5] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (eddsa). Technical Report RFC 8032, Internet Research Task Force, 2017.

[6] Hugo Krawczyk and Pasi Eronen. Hmac-based extract-and-expand key derivation function (hkdf). Technical Report RFC 5869, Internet Engineering Task Force, 2010.

[7] Adam Langley, Mike Hamburg, and Sean Turner. Elliptic curves for security. Technical Report RFC 7748, Internet Engineering Task Force, 2016.

[8] Peter L. Montgomery. Speeding the pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48(177):243–264, 1987.

[9] Stephen C. Pohlig and Martin E. Hellman. An improved algorithm for computing logarithms over GF(p) and its cryptographic significance. *IEEE Transactions on Information Theory*, 24(1):106–110, 1978.

[10] John M. Pollard. Monte carlo methods for index computation (mod p). *Mathematics of Computation*, 32(143):918–924, 1978.

[11] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[12] Victor Shoup. Lower bounds for discrete logarithms and related problems. In *Advances in Cryptology – EUROCRYPT '97*, pages 256–266, 1997.